Product Writeup.

System Architecture & Detection Approach:

To process transaction data efficiently while maintaining the deep contextual awareness required for older adult financial safety, I designed a two-stage pipeline. The first stage is a deterministic Heuristic Filter built in Pandas. It handles entity normalization (cleaning strings like "ZELLE PMT TO D VOLKOV") and applies strict mathematical rules to flag escalating transfers, rapid money mule flows, and 3-sigma out-of-profile spending deviations. The second stage utilizes LangChain and structured outputs. Flagged transactions are packaged with the user's biographical profile and sent to an LLM. The LLM acts as the final arbiter, reasoning whether a mathematically anomalous transaction is actually indicative of fraud or simply normal behavior for that specific user.

Handling False Positives and Context:

A critical requirement was avoiding patronizing false alarms. The two-stage architecture excelled here. For example, the heuristic filter flagged high-value international transactions (British Airways, Troon North Golf Club) for user Robert Chen (USR-002). However, by passing his biography to the LLM—which identifies him as an active, wealthy, traveling retired engineer—the LLM accurately dismissed these flags, resulting in a perfect 1.00 Precision score and 0 false positives across the dataset.

Tradeoffs Made:

I prioritized high precision over 100% recall. The heuristic thresholds were intentionally strict to avoid flooding the expensive LLM layer with benign data. Consequently, while the system effectively caught established exploitation and money mule behavior, it missed 8 early-stage transactions (False Negatives). I traded early detection sensitivity for operational cost efficiency and user trust.

Next Steps (With More Time):

If given more time, I would evolve this pipeline into a production-ready, event-driven architecture on AWS. First, I would deploy the Stage 1 Heuristic Filter as an AWS Lambda function processing real-time transactions from an Amazon Kinesis or SQS stream. For Stage 2, I would migrate the LangChain LLM validator to Amazon Bedrock. Using Bedrock is critical for this use case because it ensures that sensitive financial and biographical data of vulnerable adults remains private and strictly compliant with security standards, never leaving the AWS environment. To improve detection logic, I would replace the static mathematical thresholds with dynamic anomaly detection models (e.g., using Amazon SageMaker) trained on individual user categories. I would also use AWS Step

Functions to orchestrate LangGraph memory-aware agents capable of mapping multi-user transaction networks, allowing us to track complex money mule chains. Finally, I would store the user profiles in Amazon DynamoDB for low-latency retrieval during the real-time transaction evaluation.

Considerations:

I considered using an AWS infrastructure with Bedrock and Lambda. However, I prioritized simple and effective development over a cloud environment that is more difficult to maintain and takes longer to set up. I did use Langchain and Python as part of the solution stack.